

MISIÓN ROBOTECH-1

Primer contacto con la Shell y Git

RoboTech URJC

Faltan pocos minutos para el lanzamiento del cohete **RoboTech-1**. La interfaz gráfica del ordenador de control ha fallado y sólo queda operativa la terminal.
El software de la nave está almacenado en un repositorio remoto. Vuestra misión es recuperarlo, aprender a moveros por el sistema, revisar la telemetría y guardar correctamente vuestro trabajo.

Importante: en los ejercicios no se indica qué comando hay que usar. Podéis consultar la **Cheatsheet de Supervivencia** al final del documento.

No hace falta memorizar los comandos: el objetivo es aprender a **buscar qué herramienta necesitáis**.

Misión 0 — Recuperar el software de la nave

Objetivo: obtener el proyecto desde Git

El repositorio original de la nave está disponible en:

https://github.com/RoboTech-URJC/mision_robotech1_shell_26

1. Abre el enlace e inicia sesión en GitHub.
2. Pulsa **Fork** y selecciona **Create a new fork** para crear una copia del repositorio en tu cuenta.
3. En la página de tu fork, copia su URL y clónalo en tu ordenador.
4. Entra en la carpeta que se ha creado.
5. Comprueba qué archivos y carpetas contiene.
6. Muestra también los archivos y carpetas ocultos.
7. Comprueba que la carpeta forma parte de un repositorio Git.
8. Comprueba si el repositorio tiene cambios pendientes.

Cómo crear el fork

The screenshot shows a GitHub repository page for 'mision_robotech1_shell_26'. At the top, there are buttons for 'Edit Pins', 'Watch', 'Fork' (with a dropdown arrow), and 'Star'. A tooltip is visible over the 'Fork' button, stating 'You don't have any forks of this repository.' and showing a '+ Create a new fork' button. Below the repository name, there's a search bar and a list of files: 'README.md' (created 2 days ago) and 'telemetry.log' (added 2 days ago). The 'README' file is selected, showing its content. The README includes a title 'Misión RoboTech-1', a welcome message, a list of tasks to learn during the workshop, an objective, and mission norms. The right sidebar shows repository statistics: 0 stars, 0 watching, 0 forks, and 1 contributor (Amanza17).

El **fork** se crea en GitHub y queda asociado a vuestra cuenta. Después debéis clonar **vuestro fork**, no el repositorio original de RoboTech. Al clonarlo se descargan tanto los archivos como la información que Git necesita para controlar sus versiones.

Misión 1 — ¿Quién está al mando?

Objetivo: conocer vuestra sesión

1. Descubre cuál es **tu usuario**.
2. Descubre qué **otros usuarios** tienen sesión iniciada.
3. Averigua en qué **directorio** te encuentras.
4. Comprueba la **fecha y hora** del sistema.
5. Muestra los archivos y carpetas del directorio actual.
6. Averigua cuál es tu **directorio personal**.

Misión 2 — Preparar la carpeta de la misión

Objetivo: moverse y organizar archivos

1. Dentro del repositorio, crea un directorio llamado `misión_robotech`.
2. Entra dentro de él.
3. Crea dos carpetas:
 - `telemetria`
 - `informes`
4. Crea un archivo vacío llamado `checklist.txt`.
5. Comprueba que todo se ha creado correctamente.
6. Haz una copia de `checklist.txt` llamada `checklist_backup.txt`.
7. Crea un archivo llamado `prueba.txt`.
8. Elimina `prueba.txt`.

Misión 3 — Revisar los datos del cohete

Objetivo: leer y buscar información

El repositorio incluye el archivo `telemetria.log` con datos del RoboTech-1.

1. Localiza el archivo `telemetria.log`.
2. Muestra su contenido completo.
3. Muestra sólo las primeras líneas.
4. Muestra sólo las últimas líneas.
5. Busca todas las líneas que contengan `TEMP`.
6. Busca todas las líneas que contengan `WARNING`.

Opcionales:

1. Cuenta cuántas líneas de `WARNING` hay.
2. Obtén únicamente la **última** línea que contenga `TEMP`.
3. Guarda todas las líneas de `WARNING` en un archivo llamado `alertas.txt`, dentro de la carpeta `misión_robotech`.
4. Comprueba el contenido de `alertas.txt`.

Concepto nuevo

Para algunos pasos necesitarás combinar dos comandos. Busca en la cheatsheet qué significa el símbolo `|` y `>`.

Misión 4 — Variables de la misión

Objetivo: usar variables

1. Muestra el valor de la variable que contiene tu usuario.
2. Muestra el valor de la variable que contiene tu directorio personal.
3. Crea una variable llamada `COHETE` con el valor `RoboTech-1`.
4. Crea una variable llamada `DESTINO` con el valor `Marte`.
5. Muestra por pantalla una frase que utilice ambas variables.
6. Exporta la variable `COHETE`.
7. Comprueba que aparece entre las variables de entorno.

Misión 5 — Guardar el trabajo con Git

Objetivo: conocer el flujo más básico de Git

Ya habéis modificado el proyecto que descargasteis al principio.

1. Comprueba el estado del repositorio.
2. Identifica qué archivos aparecen como nuevos o modificados.
3. Prepara los cambios para guardarlos.
4. Vuelve a comprobar el estado del repositorio.
5. Crea un commit con un mensaje descriptivo.
6. Consulta el historial de commits.

Pista

En este taller sólo usaremos la idea:

cambios → preparar → commit

No es necesario subir los cambios a GitHub.

Misión final — Autorización de lanzamiento

Ya habéis utilizado todas las herramientas necesarias. Ahora debéis decidir vosotros qué comandos utilizar.

1. Entra en la carpeta `misión_robotech`.
2. Comprueba qué archivos contiene.
3. Crea un archivo llamado `informe_final.txt`.
4. Guarda dentro tu usuario y la fecha actual.

5. Añade al informe el nombre del cohete utilizando la variable `COHETE`.
6. Localiza las alertas del archivo de telemetría y añade al informe cuántas existen.
7. Copia `informe_final.txt` a la carpeta `informes`.
8. Vuelve a la raíz del repositorio.
9. Comprueba qué cambios has realizado.
10. Guarda los cambios mediante un nuevo commit.
11. Lleva tus cambios al cohete (GitHub) haciendo push.

LISTOS PARA EL DESPEGUE

Cheatsheet de Supervivencia

Recuerda: esta hoja explica las herramientas disponibles, pero no indica cuál corresponde a cada paso.

Git: obtener un proyecto

Comando	Qué hace
<code>git clone URL</code>	Descarga un repositorio remoto y crea una carpeta con el proyecto.
<code>git status</code>	Muestra el estado actual del repositorio.

Identidad y sistema

<code>whoami</code>	Muestra el usuario actual.
<code>who</code>	Muestra usuarios conectados.
<code>pwd</code>	Muestra el directorio actual.
<code>date</code>	Muestra fecha y hora.
<code>ls</code>	Lista archivos y carpetas.
<code>ls -l</code>	Lista mostrando más información.
<code>ls -a</code>	Incluye archivos y carpetas ocultos.

Moverse y gestionar archivos

<code>cd DIRECTORIO</code>	Cambia de directorio.
<code>cd ..</code>	Sube al directorio anterior.
<code>mkdir NOMBRE</code>	Crea un directorio.
<code>touch FICHERO</code>	Crea un archivo vacío.
<code>cp ORIGEN DESTINO</code>	Copia un archivo.
<code>mv ORIGEN DESTINO</code>	Mueve o renombra un archivo.
<code>rm FICHERO</code>	Elimina un archivo.
<code>rmdir DIRECTORIO</code>	Elimina un directorio vacío.

Leer y buscar

<code>cat FICHERO</code>	Muestra todo el contenido.
<code>less FICHERO</code>	Permite leer un archivo desplazándose.
<code>head FICHERO</code>	Muestra las primeras líneas.
<code>tail FICHERO</code>	Muestra las últimas líneas.
<code>grep TEXTO FICHERO</code>	Busca líneas que contienen un texto.
<code>wc -l FICHERO</code>	Cuenta líneas.
<code>file FICHERO</code>	Indica el tipo de archivo.

Combinar comandos

<code>></code>	Guarda la salida en un archivo, sobrescribiéndolo.
<code>>></code>	Añade la salida al final de un archivo.
<code> </code>	Usa la salida de un comando como entrada del siguiente.

Ejemplos genéricos:

```
comando > archivo.txt
comando >> archivo.txt
comando1 | comando2
```

Variables

```
NOMBRE="valor"
echo $NOMBRE
export NOMBRE
env
printenv
```

Variables habituales: \$USER, \$HOME, \$SHELL, \$PATH.

Git básico

git status	Muestra el estado del repositorio.
git add FICHERO	Prepara un archivo para el commit.
git add .	Prepara todos los cambios.
git commit -m "mensaje"	Guarda los cambios preparados.
git log	Muestra el historial de commits.

Git extra

git init	Convierte una carpeta normal en un nuevo repositorio Git.
git pull	Descarga e integra cambios del repositorio remoto.
git push	Sube commits al repositorio remoto.

Nota: git init, git pull y git push no son necesarios para completar este taller.

Comandos extra

Estos aparecen en las diapositivas, pero tampoco son necesarios para completar el recorrido principal:

diff A B	Compara dos archivos.
sort FICHERO	Ordena líneas.
chmod +x FICHERO	Añade permiso de ejecución.
top	Muestra procesos y estado del sistema.
htop	Monitor interactivo de procesos.
sudo COMANDO	Ejecuta con permisos de administrador.
tar	Empaqueta varios archivos.
exit	Sale de la Shell o sesión.